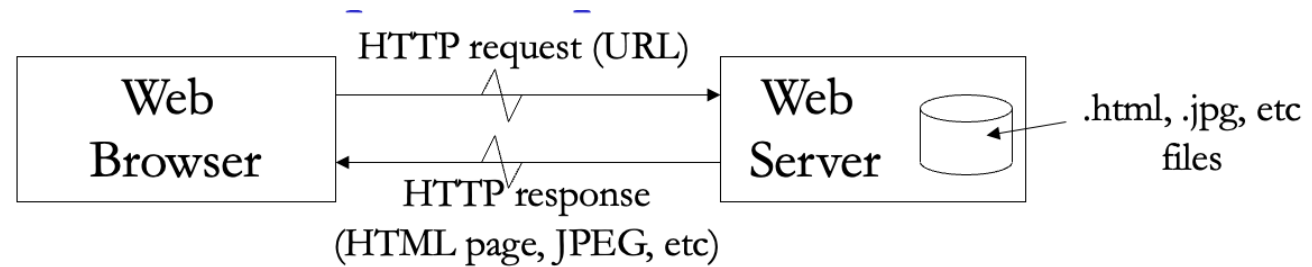


Web Platforms

For Distributed Computing

Web Pages

- In distributed computing, the term "Web" can refer to several aspects and technologies related to the World Wide Web, as it plays a significant role in the communication and interaction between distributed components and systems.
- The Web is a request-response architecture



- Originally developed to share scientific data/papers.
- HTML: HyperText Markup Language
- HTTP: HyperText Transfer Protocol
 - HyperText = document that is navigable by clicking links to jump to other parts of the document / other docs
 - Web can be viewed as one enormous hypertext document.

HTTP Behaviour

- HTTP request: is a URL requesting a file from server
 - ... plus some other stuff (referring page, browser type, etc)
 - URL format: protocol://*server.plus.domain.name*/path/to/page.html
 - » Protocol = http (or https for secure) for HTTP, ftp for FTP, etc
- HTTP response: is the requested file (or an error page)
 - ... with prefixed MIME header to indicate file type
 - » Necessary if file name doesn't indicate type (eg: ASP page)
 - Connection-oriented (ie: reliable) but temporary
 - » Connection closed after the file is retrieved - **stateless**
 - » ... thus much closer in behaviour/limitations to connectionless
 - Browser requests page, then each part (eg: images) of page
 - » Each retrieved with a separate HTTP connection

Interlude: Conn/Oriented vs Conn/less

- In TCP/IP there are two types of communication
 - Connection-oriented (eg: TCP)
 - Connection-less (eg: UDP)
- Connection-oriented:
 - Client chooses a random port and sends connection request to server on server's listening port
 - Server accepts connection and chooses a new port to communicate long-term with that particular client
 - Ports are only closed when either side disconnects
 - Communication is *reliable* – all packets must be acknowledged as received by remote end

Conn/Oriented vs Conn/less

- Connection-less:
 - Client chooses a random port and sends data packets to server on server's listening port
 - Server receives the packets and sends back response, but does not open a new port to do so
 - Thus there is no connection – server doesn't remember clients, it just assumes all clients can be treated the same
 - Communication is *unreliable* – neither client nor server attempt to ensure that all the packets were received
 - Either that, or the client / server will be told by the O/S that the packets failed to make it to the other end

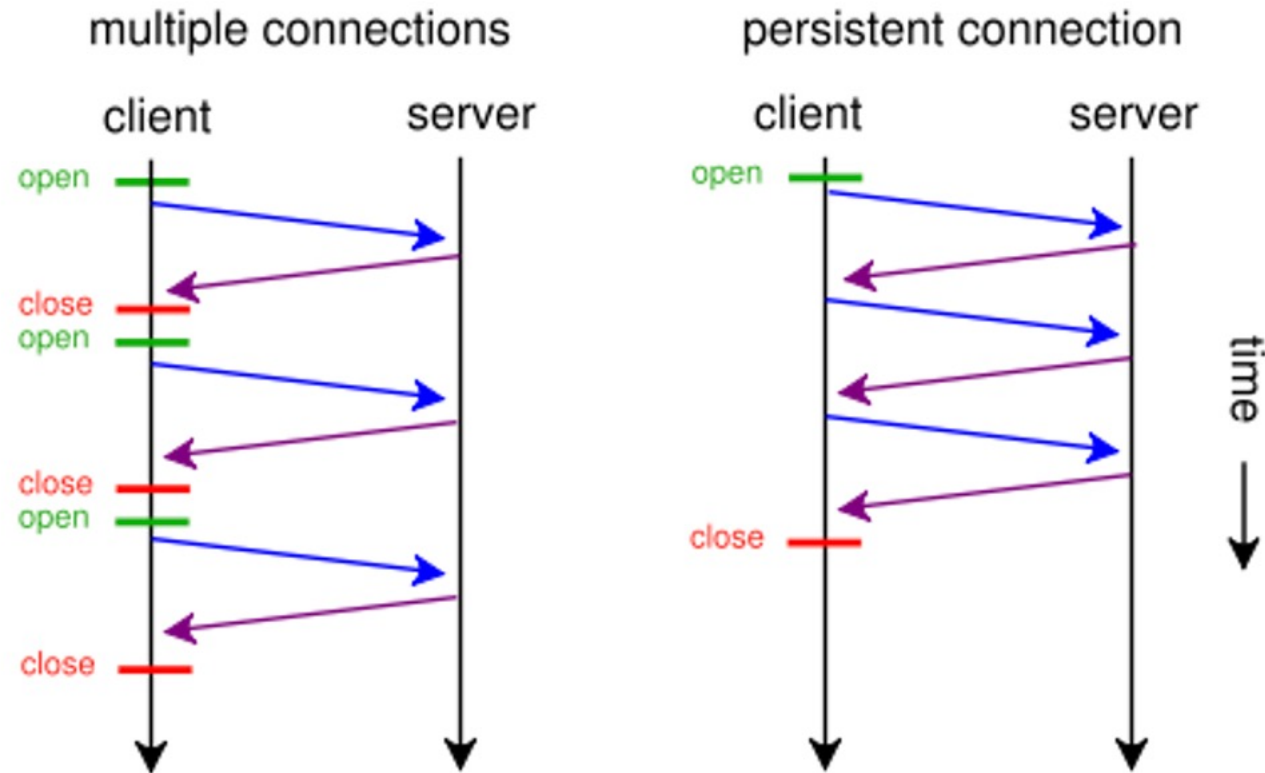
HTTP as Temp-Conn-Oriented

- HTTP is in fact connection-oriented, but temporary
 - Browser opens a reliable connection to the Web server
 - Browser sends request for page – reliable connection guarantees request will be received or error indicated
 - Web server sends back page (also reliable)
 - Browser disconnects from server regardless of whether new requests will be needed or not
- Thus HTTP is **reliable**, but there is **no long-term** unbroken communication between client and server
 - So connection-oriented, but with some of the behaviour of connection-less in that server forgets about clients

HTTP Browsing: Typical Example

1. URL entered by user (eg: typed in, or by clicking on a link)
2. Browser connects (HTTP) to URL's server on Web server port (default is 80, but can be specified in URL)
3. Requests HTML page identified in URL
4. Web server finds HTML file in filesystem, opens it and transmits it back to the browser
5. Browser parses HTML and begins to render it
 - 5.1. Browser detects that HTML contains a URL referring to another file, eg: image, stylesheet, JavaScript .js source, etc
 - 5.2. Browser opens new HTTP connection for this URL, repeating steps 2-5 above for each URL in the HTML

Connect, request packet, response packet, disconnect



Stateless vs Stateful

- Stateless client-server systems
 - The state of the server is discarded after each call by a client, regardless of whether the client continued executing
 - No on-going commitment between service and client: client can change server at will and server can forget about client after call
 - Usually arises due to the use of connectionless-style communication such as HTTP or UDP traffic
 - No ability to tell if client has terminated client-server session
 - Difficult to identify/distinguish clients; IP address+port is unique, but port always changing in connectionless systems

Stateless vs Stateful

- Stateful client-server systems
 - The state of the server is retained during the entire time that its client is active, including between client calls
 - Typical for connection-oriented (eg: TCP) systems
 - IP address+port identifies client uniquely
 - TCP notifies server if client ends session (disconnect or crash)

Stateless vs Stateful

- Where to use stateless
 - Good for simple systems whose service can be provided in a single client request (eg: Web page fetching)
 - Scalable: doesn't use resources while waiting for next client call
- Where to use stateful
 - Good for interactive systems
 - eg: Client is UI, server is processing - two pieces of the one app
 - Good for systems that require user authentication
 - Only logged in users allowed to access other server functions

Stateful from Stateless

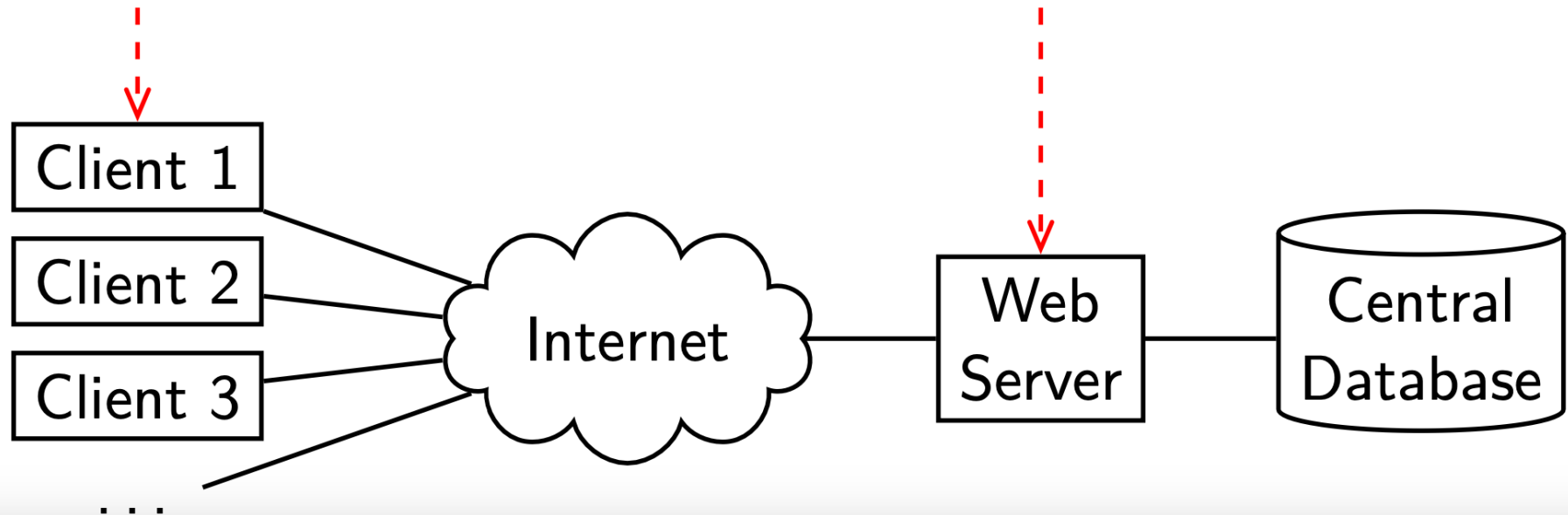
- What to do if you need stateful-ness when working with an inherently stateless protocol like HTTP?
- Can build stateful-ness on top of a stateless system
 - Must provide your own session tracking infrastructure
 - How to uniquely identify clients?
 - for Web, store client ID in cookies or pass ID in URL
 - How to detect if client has ended session?
 - Usually by having a timeout since the last access by the client
 - Alternative: Store state on client and pass back and forth between server every time the client contacts server

Web App Architecture

- ▶ Central challenge in web development: your app is slit into two parts.
- ▶ Some things happen on the client, and some on the server.

One part of your
web app here

Another part of
your web app here



Web App Architecture

The sequence of events is basically this:

1. The user enters the URL of your web site (app) into their browser.
2. The web app starts up on the server (and reads its database if needed).
3. The web app/server sends back a response, consisting of:
 - ▶ An HTML document and CSS “stylesheet(s)”;
▶ JavaScript code.
4. The JavaScript code is the *client* part, and now executes inside the user’s web browser.
5. The client periodically communicates with the server as needed.

HTML (HyperText Markup Language)

- ▶ Looks very much like XML, but technically different.
 - ▶ Both HTML and XML are derived from Standard Generalized Markup Language (SGML).
- ▶ Originally designed to be a document format.
 - ▶ And still is, really.
 - ▶ Contains elements for formatting paragraphs, lists, tables, etc.
 - ▶ Ebook formats (epub) do *now* what HTML was originally designed to do.
- ▶ Now used as the basis for web apps' user interfaces.

HTML Document Syntax

```
<!DOCTYPE html>
<html>
  <head>
    <title>My HTML Document</title>
    ...
  </head>
  <body>
    ...
  </body>
</html>
```

- ▶ `<head>...</head>` – metadata, including links to CSS and JavaScript.
- ▶ `<body>...</body>` – structure and contents.

HTML Classes and IDs

- ▶ Any element in the body can have:

- ▶ A unique ID value.

```
<div id="toolbar">...</div>
```

- ▶ Any number of (space-separated) “class” values
 - ▶ Really a kind of *tag*, unrelated to OO classes.

```
<div class="section">...</div>  
<p class="section highlighted">...</p>
```

- ▶ Both, or neither.
- ▶ These are important for:
 - ▶ Defining styles – how the document is displayed.
 - ▶ Letting your code access parts of the HTML document.
 - ▶ To query what’s displayed, or change it.

Cascading Style Sheets (CSS)

- ▶ While HTML describes document structure, CSS describes what it looks like on the screen.
- ▶ CSS “code” normally goes in a .css file, linked to from the HTML document:

```
<html>
  <head>
    <link rel="stylesheet"
          type="text/css" href="my_style.css"/>
    ...
  </head>
  <body>...</body>
</html>
```

- ▶ The web browser then asks the server to give it my_style.css.
- ▶ (You can also embed CSS directly inside the HTML.)

CSS Syntax

- ▶ CSS files consist of several *rules*.
- ▶ Each rule has:
 - ▶ A *selector*, an expression that identifies part(s) of the HTML/DOM.
 - ▶ One or more *declarations*, which specify values for various properties – fonts, colour, size, etc.

```
selector {  
    property: value;  
}
```

```
selector {  
    property: value;  
    property: value;  
}
```

```
...
```

CSS Selectors

- ▶ You can select elements by tag name, ID, class, and other attributes.

```
* {...}           // Select everything
p {...}           // Select all paragraphs.
#toolbar {...}    // Select element with id="toolbar".
.section {...}    // Select all elements with
                  // class="section ...".
[type="text"] {...} // Elements with type="text".
```

- ▶ We can combine these (no spaces!):

```
p.section {...}           // <p class="section ...">
input[type="text"] {...}  // <input type="text">
.section.highlighted {...} // Multiple classes
.section[x="5"][y="10"]   // Class & attributes
```

- ▶ *#id* is already unique, so no point combining with others.

CSS Selector Combinators

- ▶ Selectors can also look at the tree structure:

```
p .section {...} // Example A
```

```
.section[x="5"] *[type="text"] {...} // Example B
```

- ▶ Two selectors separated by a space – matches the second anywhere within the first.
- ▶ Example A: searches for `<p>` first, *then* searches for `class="section ..."` within those particular paragraphs only.
- ▶ Example B: finds elements with `type="text"` *within* elements with `class="section ..."` and `x="5"`.
- ▶ Can also perform a simple “or” operation:

```
p, .section {...}
```

- ▶ Two selectors separated by a comma – matches either one.

CSS Selectors: Pseudo-Classes and Pseudo-Elements

- ▶ “Pseudo-class” selectors identify elements by their state:

```
a:visited {...} // Select links (<a> elements) that  
                // have been visited.
```

- ▶ “Pseudo-elements” can be created and added into the document by CSS.
 - ▶ ::before or ::after existing elements.
 - ▶ Purely for stylistic purposes.

```
// Selects/creates a new pseudo-elements before  
// each .section element.  
.section::before {  
    content: "Welcome to a new section...";  
    ...  
}
```

CSS Declarations

There are a huge number of properties. Some include:

- ▶ `border: thickness style colour;`
`border-position: thickness style colour;`

```
border: 2px solid red;  
border-left: 1px dashed gray;
```

- ▶ `font-family: font [ backup-font ... ];`

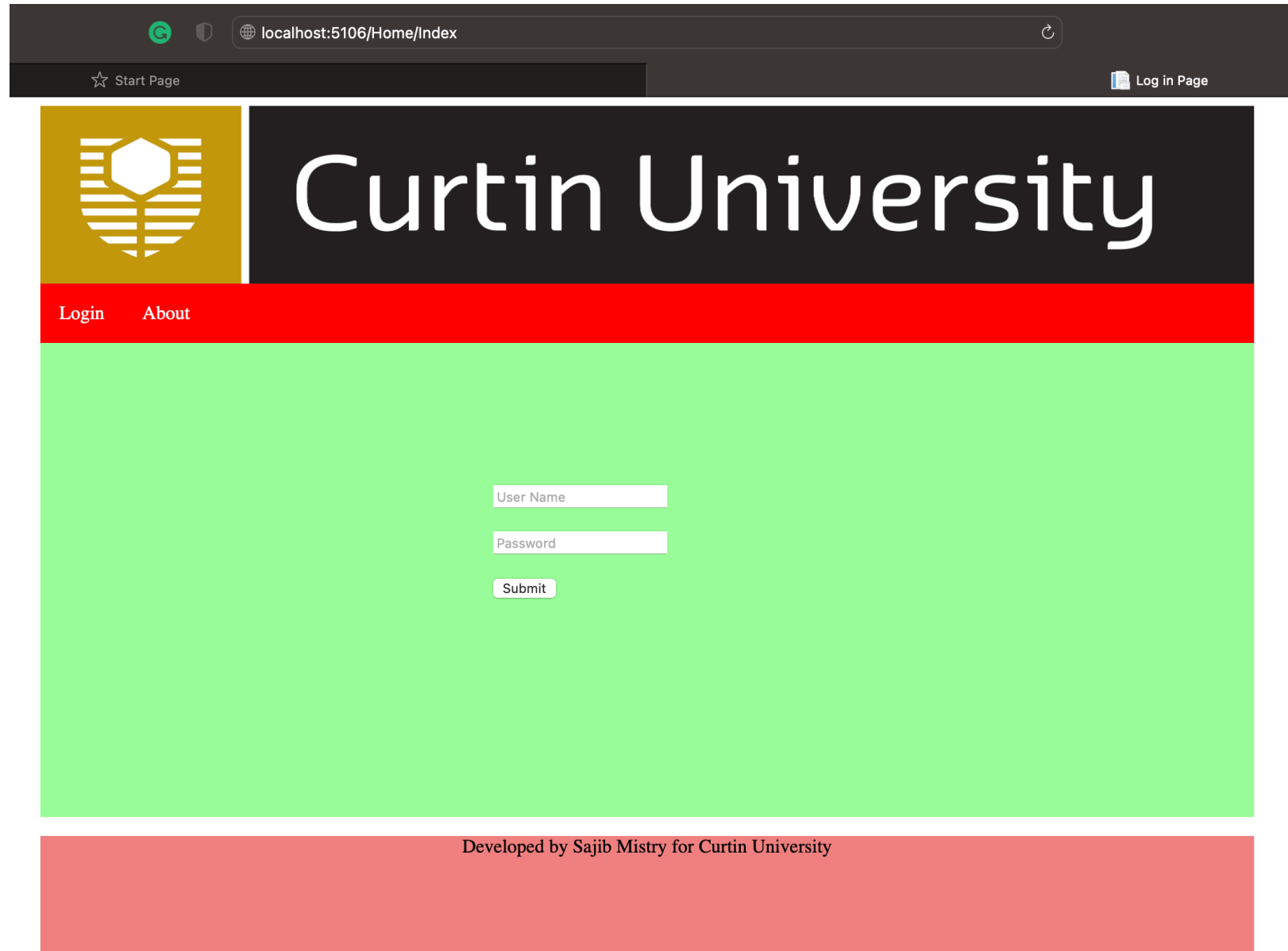
```
font-family: monospace;
```

```
font-family: Ariel Helvetica sans-serif;
```

- ▶ `position: static / absolute / fixed / relative / sticky / ...;`
`width: distance;` `height: distance;`
`left: distance;` `top: distance;`

- ▶ Where and how big should the element(s) be?

HTML + CSS example



The image shows a web browser window with the address bar displaying "localhost:5106/Home/Index". The browser's address bar includes a green "G" icon, a shield icon, and a refresh button. Below the address bar, there is a "Start Page" link with a star icon and a "Log in Page" link with a document icon.

The main content area of the browser displays a login page for Curtin University. The page has a dark blue header with the Curtin University logo on the left and the text "Curtin University" on the right. Below the header is a red navigation bar with the links "Login" and "About". The main body of the page is light blue and contains a login form with the following elements:

- A "User Name" input field.
- A "Password" input field.
- A "Submit" button.

At the bottom of the page, there is a dark blue footer with the text "Developed by Sajib Mistry for Curtin University".

Index.html + site.css

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Log in Page</title>
  <link rel="stylesheet" href="/css/site.css">
</head>
<body>
  <div id="header">
  </div>
  <div id="navigation">
    <ul>
      <li><a href="/Home/Index">Login</a></li>
      <li><a href="/Home/About">About</a></li>
    </ul>
  </div>
  <div id="main">
    <div id="LoginForm">
      <form action="/Home/" method="post">
        <input type="text" id="Sname" name="SUsername" placeholder="User Name" />
        <input type="password" id="SPass" name="SPassword" placeholder="Password" />
        <input type="submit" value="Submit">
      </form>
    </div>
  </div>
  <div id="footer">
    <p> Developed by Sajib Mistry for Curtin University</p>
  </div>
</body>
</html>
```

```
body {
  width: 1024px;
  margin-left: auto;
  margin-right: auto;
}

div#header {
  background-image: url(../images/Curtin_0.png);
  background-size: 1024px 150px;
  height: 150px;
}

div#navigation ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
  overflow: hidden;
  background-color: red;
}

div#navigation li {
  float: left;
}

div#navigation li a {
  display: block;
  color: white;
  text-align: center;
  padding: 16px;
  text-decoration: none;
}

div#navigation li a:hover {
  background-color: #111111;
}
```

Index.html + site.css

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Log in Page</title>
  <link rel="stylesheet" href="/css/site.css">
</head>
<body>
  <div id="header">
  </div>
  <div id="navigation">
    <ul>
      <li><a href="/Home/Index">Login</a></li>
      <li><a href="/Home/About">About</a></li>
    </ul>
  </div>
  <div id="main">
    <div id="LoginForm">
      <form action="/Home/" method="post">
        <input type="text" id="Sname" name="SUname" placeholder="User Name" />
        <input type="password" id="SPass" name="SPassword" placeholder="Password" />
        <input type="submit" value="Submit">
      </form>
    </div>
  </div>
  <div id="footer">
    <p> Developed by Sajib Mistry for Curtin University</p>
  </div>
</body>
</html>
```

```
div#main {
  background-color: palegreen;
  height: 400px;
}

div#LoginForm {
  width: 300px;
  padding-top: 100px;
  margin-left: auto;
  margin-right: auto;
}

input {
  display: block;
  margin: 20px 20px 20px 20px;
}

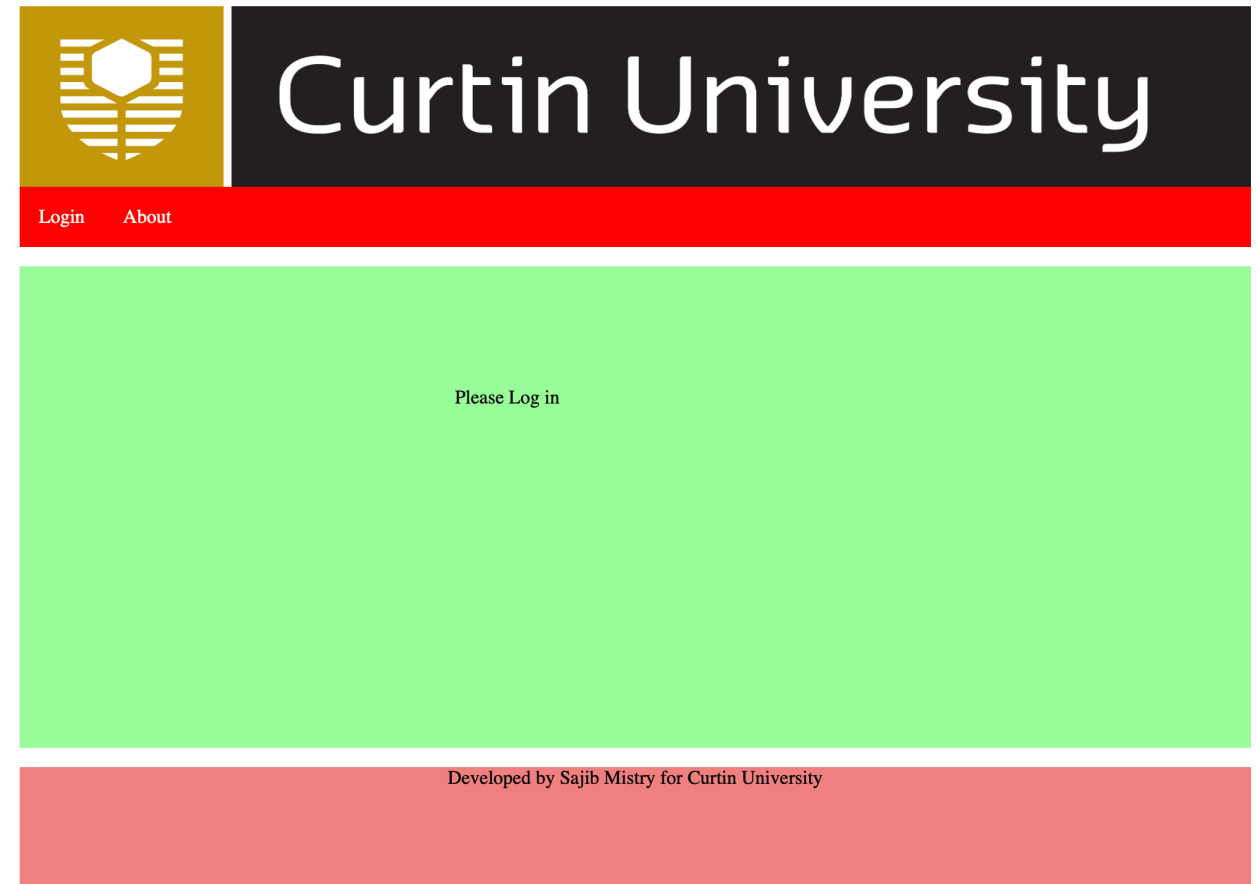
p#status {
  width: 300px;
  padding-top: 100px;
  margin-left: auto;
  margin-right: auto;
}

button {
  margin: 20px 20px 20px 20px;
}

div#footer {
  background-color: lightcoral;
  text-align: center;
  height: 100px;
}
```

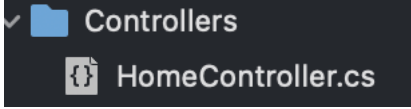
About.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Log in Page</title>
  <link rel="stylesheet" href="/css/site.css">
</head>
<body>
  <div id="header">
  </div>
  <div id="navigation">
    <ul>
      <li><a href="/Home/Index">Login</a></li>
      <li><a href="/Home/About">About</a></li>
    </ul>
  </div>
  <div id="main">
    <p id="status"> Please Log in </p>
  </div>
  <div id="footer">
    <p> Developed by Sajib Mistry for Curtin University</p>
  </div>
</body>
</html>
```

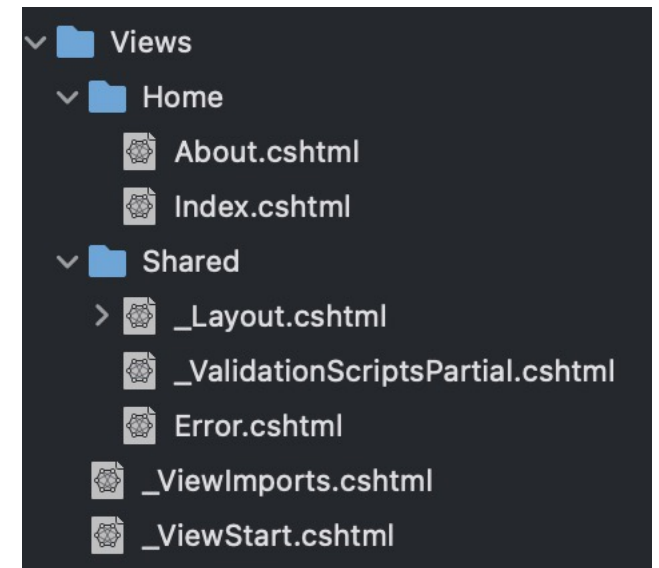
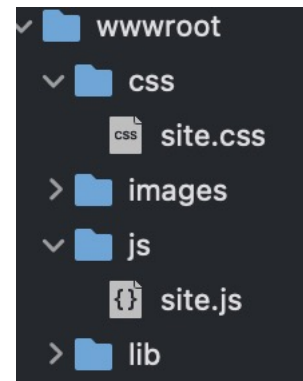


Both Index and About is sharing a layout?

Server-side scripting

- We will follow ASP.NET core (Model-View-Controller)
- <http://localhost:port/> lands to 
- Home controller has two methods – index and about
 - Index method is triggered by the endpoint localhost or localhost/home/index
 - About method is triggered by localhost/home/about
- Each method has a View() that renders HTML from corresponding View folders
 - Index → view()->index.cshtml->_layout.cshtml

- CSS and JS files are in wwwroot



Home controller.cs

```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    public IActionResult Index()
    {
        return View();
    }

    [HttpGet]
    public IActionResult Index(int sessionId)
    {
        if(sessionId== 54353454)
        {
            ViewBag.Message = "Logged in";
        }
        return View();
    }

    [HttpPost]
    public IActionResult Index(string SName, string SPass)
    {
        if(SName.Equals("sajib") && SPass.Equals("mistry"))
        {
            ViewBag.Message = "Logged in";
        }
        else
        {
            ViewBag.Error = "User name and Password are not matched. Try again";
        }
        return View();
    }
}
```

```
public IActionResult About()
{
    return View();
}

[HttpGet]
public IActionResult About(int sessionId)
{
    if (sessionId == 54353454)
    {
        ViewBag.Message = "Logged in";
    }
    return View();
}
```

ViewBag

- ViewBag is a dynamic property in ASP.NET MVC that allows you to store data and pass it from a controller to a view. It uses dynamic typing, so you can assign any type of data to it, and the type is determined at runtime. Here's how you can use ViewBag in a controller and access it in a view:
- In the controller:
- In the corresponding view (Razor syntax):

```
public ActionResult Index()
{
    ViewBag.Message = "Hello, ViewBag!";
    return View();
}
```

```
@ViewBag.Message
```

ViewData

- ViewData is similar to ViewBag in that it allows you to pass data from a controller to a view, but it uses a ViewDataDictionary to store data as a key-value pair collection. Unlike ViewBag, which uses dynamic typing, ViewData requires typecasting when retrieving data in the view.
- Controller:

```
public ActionResult Index()
{
    ViewData["Message"] = "Hello, ViewData!";
    return View();
}
```

View:

```
@(string)ViewData["Message"]
```

Embedding C# Code in html

- In a CSHTML (C# Razor) file, the @ symbol is used to transition from HTML markup to C# code. It's a special character that indicates to the Razor view engine that you are switching from rendering HTML to executing C# code within the view.

```
@{  
    var greeting = "Hello, World!";  
}  
<p>@greeting</p>
```

```
<p>The current date and time is:  
@DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss")</p>
```

```
@if (condition)  
{  
    <p>This will be displayed if the condition is true.</p>  
}
```

```
@foreach (var item in Model.Items)  
{  
    <p>@item.Name</p>  
}
```

_Layout.cshtml

```
@{
    string message = ViewBag.message;
}

<!DOCTYPE html>
<html lang="en">
<head>
    <title>@ViewBag.Title</title>
    <link rel="stylesheet" href="~/css/site.css">
</head>
<body>
    <div id="header">
    </div>
    <div id="navigation">
        @if (message != null && message.Equals("Logged in"))
        {
            <ul>
                <li>
                    <a href="/Home/Index?sessionId=54353454">Login</a>
                </li>
                <li>
                    <a href="/Home/About?sessionId=54353454">About</a>
                </li>
            </ul>
        }
        else
        {
            <ul>
                <li><a href="/Home/Index">Login</a></li>
                <li><a href="/Home/About">About</a></li>
            </ul>
        }
    </div>
```

```
        <div id="main">
            @RenderBody()
        </div>
        <div id="footer">
            <p> Developed by Sajib Mistry for Curtin University</p>
        </div>
    </body>
</html>
```

In ASP.NET MVC and Razor Pages, the `@RenderBody()` method is a special directive used within a layout file (usually with a `.cshtml` extension) to indicate where the content of a specific view should be rendered.

Index.cshtml

```
@{
    ViewData["Title"] = "Log in Page";
    string message = ViewBag.message;
    string error = ViewBag.Error;
}

@if(message!=null && message.Equals("Logged in"))
{
    <p id="status">User Successfully logged in</p>
}
else
{
    <div id="LoginForm">
        @if (error != null && error.Length > 0)
        {
            <p>@error</p>
        }
        <form action="/Home/" method="post">
            <input type="text" id="Sname" name="SUsername" placeholder="User Name" />
            <input type="password" id="SPass" name="SPassword" placeholder="Password" />
            <input type="submit" value="Submit">
        </form>
    </div>
}
```

HTML Form

- An HTML form is a crucial element for gathering user input on a web page. It allows users to enter data and submit it to a web server for processing. Forms are a fundamental part of web development and are used for various purposes, such as user registration, search, login, contact forms, and more.
- The `<form>` element defines the container for the input fields.
- The action attribute specifies where the form data should be sent when the user submits the form. In this case, it's set to `"/home/index"`.
- The method attribute specifies the HTTP method to be used for submitting the form data. Common methods are GET and POST.
- Inside the form, there are several input elements:
 - `<input type="text">` for the user's name.
 - `<input type="email">` for the user's email address.
 - `<textarea>` for the user's message.
 - `<input type="submit">` submits a post request to the server specified by the action.

About.csthml

```
@{
    ViewData["Title"] = "About";
    string message = ViewBag.message;
}

@if (message != null && message.Equals("Logged in"))
{
    <p id="status">In software development, "distributed" often refers to systems or applications
    that are spread across multiple locations or nodes.
    A "lab" in this context might be a test environment where developers work on distributed systems.
    This could involve creating, testing, and optimizing software for distributed computing or networked applications.</p>
}
else
{
    <p id="status"> Please Log in </p>
}
```

- Now demo time!!
- Check the attached source code
- Next week, JS and Ajax to call remote APIs.